Database Systems Lab — Week 03 Manual

Advanced SQL: Aggregations & Analytics

Course: Database Systems Lab (Data Science Program)

Duration: 3 hours (2 lab sessions)

Prerequisites: Lab 02 complete — e-commerce dataset loaded, 10 queries done, GitHub repo updated

---

Learning Objectives

By the end of this lab you will be able to:

- Summarise data using aggregate functions: COUNT, SUM, AVG, MIN, MAX
- Group results with GROUP BY and filter groups with HAVING
- Understand the difference between WHERE and HAVING and when to use each
- Use window functions (ROW_NUMBER, RANK, LAG, LEAD) for analytical queries
- Write Common Table Expressions (CTEs) to break complex queries into readable steps
- Apply these skills to time-series and cohort-style analysis on the existing dataset
- Commit analytical queries to GitHub as reusable SQL scripts

---

Why This Matters for Data Science

Last week you learned how to retrieve and filter rows. That is the foundation. But real data science work is almost never about individual rows — it is about patterns across groups, trends over time, and rankings within categories.

Consider what a data analyst at an e-commerce company actually asks day to day: Which product category generates the most revenue? How does monthly sales compare to the previous month? Which customers are in the top 10% by spending? None of those questions can be answered by WHERE and SELECT alone. They require aggregation, grouping, and analytical functions — which is exactly what this lab covers.

The skills here map directly to what your projects will need in Weeks 9–14. When you are building a recommendation engine or a churn prediction model, the features you feed into your ML model will come from exactly these kinds of queries.

Part 1 — Dataset Extension

You already have the e-commerce schema from Week 2: customers, products, orders, order_items. This week you will extend it with one new table — **user_sessions** — that simulates website engagement data. This gives you time-series and behavioural data to practice on, which is far more representative of real analytics work than a simple orders table alone.

Open your terminal, connect to PostgreSQL, and run the following.

```
sudo service postgresql start        # WSL/Linux users
psql -U your_username -d lab1_db
```

Create a new setup file first:

```
# In a second terminal tab
cd ~/database-labs/lab2
nano lab3_extension.sql
```

Paste the following:

```
-- ============================================================
-- Lab 03: Dataset Extension — User Sessions
-- Author: [Your Name]
-- Date:   [Today's Date]
-- ============================================================

DROP TABLE IF EXISTS user_sessions;

CREATE TABLE user_sessions (
    session_id      SERIAL PRIMARY KEY,
    customer_id     INTEGER REFERENCES customers(customer_id),
    session_date    DATE NOT NULL,
    pages_viewed    INTEGER NOT NULL DEFAULT 1,
    duration_mins   DECIMAL(6,2) NOT NULL,
    device          VARCHAR(20) NOT NULL,
    converted       BOOLEAN NOT NULL DEFAULT FALSE
);

INSERT INTO user_sessions
    (customer_id, session_date, pages_viewed, duration_mins, device, converted)
VALUES
-- Ali Hassan (customer 1)
(1, '2024-10-05', 8,  14.5, 'mobile',  FALSE),
(1, '2024-10-18', 12, 22.3, 'desktop', TRUE),
(1, '2025-01-15', 5,   8.1, 'mobile',  FALSE),
(1, '2025-04-01', 15, 31.0, 'desktop', TRUE),
-- Sara Khan (customer 2)
(2, '2024-09-10', 3,   4.2, 'mobile',  FALSE),
(2, '2024-09-11', 10, 18.7, 'desktop', TRUE),
(2, '2025-06-15', 6,   9.5, 'tablet',  FALSE),
(2, '2025-06-17', 14, 25.4, 'desktop', TRUE),
-- Usman Tariq (customer 3)
(3, '2024-09-28', 7,  12.0, 'mobile',  FALSE),
(3, '2024-09-30', 18, 35.6, 'desktop', TRUE),
(3, '2025-04-12', 9,  16.2, 'mobile',  FALSE),
(3, '2025-04-14', 11, 20.8, 'desktop', TRUE),
-- Fatima Malik (customer 4)
```

```
(4, '2024-10-10', 4,   6.3, 'tablet',  FALSE),
(4, '2024-10-14', 13, 24.1, 'desktop', TRUE),
(4, '2025-02-20', 7,  11.5, 'mobile',  FALSE),
-- Bilal Ahmed (customer 5)
(5, '2024-10-30', 6,  10.2, 'mobile',  FALSE),
(5, '2024-10-31', 9,  17.4, 'desktop', TRUE),
(5, '2025-04-28', 5,   8.8, 'mobile',  FALSE),
(5, '2025-04-30', 12, 23.5, 'desktop', TRUE),
-- Ayesha Noor (customer 6)
(6, '2024-11-15', 20, 42.0, 'desktop', TRUE),
(6, '2025-08-12', 25, 55.0, 'desktop', TRUE),
-- Hamza Sheikh (customer 7)
(7, '2024-12-01', 8,  13.7, 'mobile',  FALSE),
(7, '2024-12-04', 11, 20.0, 'desktop', TRUE),
-- Zainab Iqbal (customer 8)
(8, '2024-12-18', 3,   5.5, 'mobile',  FALSE),
(8, '2024-12-19', 7,  12.3, 'tablet',  FALSE),
(8, '2024-12-20', 16, 30.1, 'desktop', TRUE),
(8, '2025-07-03', 9,  14.2, 'mobile',  FALSE),
(8, '2025-07-04', 13, 22.6, 'desktop', TRUE),
-- Farhan Butt (customer 9)
(9, '2025-01-05', 11, 19.8, 'desktop', FALSE),
(9, '2025-01-07', 17, 33.4, 'desktop', TRUE),
(9, '2025-08-25', 8,  15.0, 'mobile',  FALSE),
(9, '2025-08-27', 22, 44.7, 'desktop', TRUE),
-- Sana Rashid (customer 10)
(10,'2025-01-20', 6,  10.5, 'tablet',  FALSE),
(10,'2025-01-21', 14, 26.8, 'desktop', TRUE),
(10,'2025-07-10', 10, 18.3, 'mobile',  FALSE),
(10,'2025-07-11', 18, 37.2, 'desktop', TRUE),
-- Omar Hussain (customer 11)
(11,'2025-02-01', 5,   8.0, 'mobile',  FALSE),
(11,'2025-02-02', 12, 22.5, 'desktop', TRUE),
-- Nadia Aziz (customer 12)
(12,'2025-02-12', 3,   4.8, 'mobile',  FALSE),
(12,'2025-07-18', 8,  13.5, 'mobile',  FALSE),
(12,'2025-07-19', 15, 28.9, 'desktop', TRUE),
-- Tariq Mehmood (customer 13)
(13,'2025-02-28', 22, 48.3, 'desktop', TRUE),
-- Hina Qureshi (customer 14)
(14,'2025-03-08', 7,  11.0, 'tablet',  FALSE),
(14,'2025-03-09', 16, 30.5, 'desktop', TRUE),
(14,'2025-07-30', 4,   6.7, 'mobile',  FALSE),
(14,'2025-08-01', 10, 19.4, 'desktop', FALSE),
-- Kamran Javed (customer 15)
(15,'2025-03-20', 5,   9.2, 'mobile',  FALSE),
(15,'2025-03-21', 9,  16.8, 'desktop', FALSE),
-- Rabia Siddiqui (customer 16)
(16,'2025-05-06', 6,  10.0, 'mobile',  FALSE),
(16,'2025-05-07', 18, 36.2, 'desktop', TRUE),
-- Imran Chaudhry (customer 17)
(17,'2025-05-10', 28, 58.0, 'desktop', TRUE),
-- Maira Aslam (customer 18)
(18,'2025-05-18', 7,  12.4, 'mobile',  FALSE),
(18,'2025-05-20', 13, 24.7, 'desktop', TRUE),
-- Asad Nawaz (customer 19)
(19,'2025-05-30', 9,  15.8, 'mobile',  FALSE),
(19,'2025-06-01', 21, 43.2, 'desktop', TRUE),
-- Kiran Baig (customer 20)
(20,'2025-06-08', 4,   7.3, 'mobile',  FALSE),
(20,'2025-06-10', 11, 20.1, 'desktop', FALSE);
```

Load it into psql:

```
\i /home/your_username/database-labs/lab2/lab3_extension.sql
```

Verify:

```
SELECT COUNT(*) FROM user_sessions;
SELECT * FROM user_sessions LIMIT 5;
```

You should have around 56 session records. Now commit before writing a single query:

```
cd ~/database-labs
git add lab2/lab3_extension.sql
git commit -m "Lab 3: Added user_sessions table for analytics exercises"
git push origin main
```

Part 2 — Core Concepts

2.1 Aggregate Functions

Aggregate functions collapse many rows into a single summary value. They are the foundation of everything from a sales report to a feature engineering pipeline.

```
-- COUNT: how many rows
SELECT COUNT(*) AS total_orders FROM orders;

-- COUNT on a column: counts non-NULL values only
SELECT COUNT(shipped_date) AS shipped_orders FROM orders;

-- SUM: total value
SELECT SUM(total_amount) AS total_revenue FROM orders;

-- AVG: average value
SELECT AVG(total_amount) AS avg_order_value FROM orders;

-- MIN and MAX: extremes
SELECT MIN(price) AS cheapest,
       MAX(price) AS most_expensive
FROM products;

-- All together
SELECT COUNT(*)              AS total_orders,
       SUM(total_amount)     AS total_revenue,
       AVG(total_amount)     AS avg_order_value,
       MIN(total_amount)     AS smallest_order,
       MAX(total_amount)     AS largest_order
FROM orders
WHERE status = 'delivered';
```

Run that last query. You now have a complete summary of your delivered orders in one shot. In a BI dashboard, this is what populates the KPI cards at the top of the screen.

2.2 GROUP BY

Aggregates become powerful when you group them. GROUP BY tells PostgreSQL to apply the aggregate function separately to each group of rows that share the same value in a column.

```
-- Revenue by order status
SELECT status,
       COUNT(*)          AS num_orders,
       SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;

-- Average session duration by device
SELECT device,
       COUNT(*)               AS sessions,
       ROUND(AVG(duration_mins), 2) AS avg_duration_mins
FROM user_sessions
GROUP BY device
ORDER BY avg_duration_mins DESC;
```

The rule is: every column in SELECT must either be inside an aggregate function or appear in the GROUP BY clause. If you put a column in SELECT that is in neither, PostgreSQL will give you an error. This trips up nearly every beginner — when you hit it, remember this rule.

```
-- WRONG — product_name is not in GROUP BY and not aggregated
SELECT product_name, category, COUNT(*)
FROM products
GROUP BY category;

-- CORRECT
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category;

-- CORRECT — if you want product_name, include it in GROUP BY too
SELECT category, product_name, COUNT(*)
FROM products
GROUP BY category, product_name;
```

2.3 HAVING — Filtering Groups

WHERE filters individual rows before grouping. HAVING filters groups after aggregation. This is one of the most commonly confused distinctions in SQL.

```
-- WHERE filters rows, then GROUP BY groups, then HAVING filters groups
SELECT status,
       COUNT(*)          AS num_orders,
       SUM(total_amount) AS total_revenue
FROM orders
WHERE order_date >= '2025-01-01'     -- filter rows first
GROUP BY status
HAVING COUNT(*) > 3                  -- then filter groups
ORDER BY total_revenue DESC;
```

Read that query as: "From orders placed in 2025 or later, group them by status, and only show me statuses that have more than 3 orders."

Another example — find product categories where average price is above 3000:

```
SELECT category,
       COUNT(*)                      AS products,
       ROUND(AVG(price), 2)          AS avg_price
FROM products
GROUP BY category
HAVING AVG(price) > 3000
ORDER BY avg_price DESC;
```

You cannot use a column alias in HAVING for the same reason you cannot use one in WHERE — both run before SELECT assigns the alias. Repeat the expression: `HAVING AVG(price) > 3000`, not `HAVING avg_price > 3000`.

2.4 Window Functions

This is where SQL goes from useful to genuinely powerful for data science. Window functions perform a calculation across a set of rows that are related to the current row — without collapsing them into a single group like GROUP BY does. You keep all your rows and add a new computed column alongside them.

```
-- ROW_NUMBER: assign a sequential number to each row within a group
SELECT customer_id,
       order_date,
       total_amount,
       ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY total_amount DESC
       ) AS order_rank_by_customer
FROM orders;
```

Read the OVER clause as: "For each row, look at all rows that share the same customer_id, sort those by total_amount descending, and assign a row number." Customer 1's orders get numbered 1, 2, 3 among themselves. Customer 2's orders also start from 1. Each customer has their own independent ranking.

```
-- RANK: like ROW_NUMBER but ties get the same rank
-- (two orders with equal amount both get rank 1, next is rank 3)
SELECT customer_id,
       total_amount,
       RANK() OVER (ORDER BY total_amount DESC) AS overall_rank
FROM orders;

-- DENSE_RANK: ties get the same rank, but no gaps in numbering
-- (two at rank 1 means next is rank 2, not rank 3)
SELECT customer_id,
       total_amount,
       DENSE_RANK() OVER (ORDER BY total_amount DESC) AS dense_rank
FROM orders;
```

LAG and LEAD — looking at previous and next rows:

This is the function data scientists use for time-series feature engineering. LAG looks at the previous row, LEAD looks at the next row, within a defined window.

```
-- Month-over-month revenue comparison using LAG
SELECT TO_CHAR(order_date, 'YYYY-MM')    AS month,
       SUM(total_amount)                  AS revenue,
       LAG(SUM(total_amount)) OVER (
           ORDER BY TO_CHAR(order_date, 'YYYY-MM')
       )                                  AS prev_month_revenue
FROM orders
WHERE status = 'delivered'
GROUP BY TO_CHAR(order_date, 'YYYY-MM')
ORDER BY month;
```

The prev_month_revenue column will be NULL for the first month (there is nothing before it) and then show each month's revenue from the row above. This is exactly the pattern used to compute month-over-month growth in any analytics dashboard.

2.5 Common Table Expressions (CTEs)

A CTE is a named, temporary result set you define at the top of a query and then reference below. Think of it as giving a subquery a name so you can read your SQL top to bottom like a recipe, rather than inside-out like a puzzle.

```
-- Without CTE: hard to read, nested subquery
SELECT customer_id, total_spent
FROM (
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    WHERE status = 'delivered'
    GROUP BY customer_id
) AS customer_totals
WHERE total_spent > 10000;

-- With CTE: reads naturally, same result
WITH customer_totals AS (
    SELECT customer_id,
           SUM(total_amount) AS total_spent
    FROM orders
    WHERE status = 'delivered'
    GROUP BY customer_id
)
SELECT customer_id, total_spent
FROM customer_totals
WHERE total_spent > 10000
ORDER BY total_spent DESC;
```

You can chain multiple CTEs together, each building on the previous one:

```
WITH
-- Step 1: Calculate each customer's total spend
customer_spend AS (
    SELECT customer_id,
           SUM(total_amount)  AS total_spent,
           COUNT(*)           AS num_orders
    FROM orders
    WHERE status = 'delivered'
    GROUP BY customer_id
),
-- Step 2: Classify customers by spend tier
customer_tiers AS (
    SELECT customer_id,
           total_spent,
           num_orders,
           CASE
               WHEN total_spent > 30000 THEN 'VIP'
               WHEN total_spent > 10000 THEN 'Regular'
               ELSE 'Occasional'
           END AS tier
    FROM customer_spend
)
-- Step 3: Count how many customers fall into each tier
SELECT tier,
       COUNT(*)           AS num_customers,
       SUM(total_spent)   AS total_revenue
FROM customer_tiers
GROUP BY tier
ORDER BY total_revenue DESC;
```

This three-step query is something you would find in a real customer analytics report. Without CTEs it would be deeply nested and nearly unreadable. With CTEs, each step is isolated, named, and testable independently.

Part 3 — Lab Exercises

Create a new file for this week's queries:

```
cd ~/database-labs
mkdir lab3
nano lab3/queries.sql
```

Save every query you write into this file as you go. At the end of the lab you will commit the whole thing.

Query 1 — Overall business summary

Write a single query that gives the total number of orders, total revenue, average order value, and the size of the largest single order — but only for delivered orders.

```
SELECT COUNT(*)                          AS total_orders,
       SUM(total_amount)                 AS total_revenue,
       ROUND(AVG(total_amount), 2)       AS avg_order_value,
       MAX(total_amount)                 AS largest_order
FROM orders
WHERE status = 'delivered';
```

This is a KPI summary. Every analytics dashboard starts with something like this. Look at the numbers — do they make sense given what you know about the data from last week?

Query 2 — Revenue by product category

Find how much revenue each product category has generated across all order items. Join order_items to products to get the category, then aggregate.

```
SELECT p.category,
       COUNT(DISTINCT oi.order_id)        AS orders_containing,
       SUM(oi.quantity * oi.unit_price)   AS category_revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY category_revenue DESC;
```

Note the JOIN here — you need product information (category) that lives in one table, and sales data (quantity, unit_price) that lives in another. Joining tables and then aggregating is one of the most common patterns in analytics. You will cover JOINs deeply in Week 4, but this gives you early exposure.

Query 3 — Monthly order volume and revenue

Show how many orders were placed and total revenue generated per month. Format the month as 'YYYY-MM' for clean sorting.

```
SELECT TO_CHAR(order_date, 'YYYY-MM')    AS month,
       COUNT(*)                           AS num_orders,
       SUM(total_amount)                  AS monthly_revenue
FROM orders
GROUP BY TO_CHAR(order_date, 'YYYY-MM')
ORDER BY month;
```

This is a time-series aggregation. Look at the results — can you spot which months were busiest? Notice how August 2025 looks. This kind of query is the first thing anyone runs when investigating sales trends.

Query 4 — HAVING: active cities

Find all cities where customers placed more than 2 orders total. Show the city, customer count, and order count.

```
SELECT c.city,
       COUNT(DISTINCT c.customer_id)   AS customers,
       COUNT(o.order_id)               AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city
HAVING COUNT(o.order_id) > 2
ORDER BY total_orders DESC;
```

Try removing the HAVING clause and see all cities. Then add it back. Notice the difference between filtering with WHERE (which runs before grouping) and filtering with HAVING (which runs after).

Query 5 — Device performance with HAVING

From user_sessions, find which devices have an average session duration above 15 minutes. Only include devices that have been used for at least 5 sessions.

```
SELECT device,
       COUNT(*)                            AS total_sessions,
       ROUND(AVG(duration_mins), 2)        AS avg_duration,
       SUM(CASE WHEN converted THEN 1 ELSE 0 END) AS conversions,
       ROUND(
           100.0 * SUM(CASE WHEN converted THEN 1 ELSE 0 END) / COUNT(*),
           1
       )                                   AS conversion_rate_pct
FROM user_sessions
GROUP BY device
HAVING COUNT(*) >= 5
   AND AVG(duration_mins) > 15
ORDER BY conversion_rate_pct DESC;
```

The `SUM(CASE WHEN converted THEN 1 ELSE 0 END)` pattern is very common — it counts how many rows in a group satisfy a condition. You are essentially doing a conditional count. Look at the conversion rates by device — which device converts best? This is the kind of question a product team asks every week.

Query 6 — Ranking orders with ROW_NUMBER

For each customer, rank their orders from largest to smallest. Show all orders with the rank assigned per customer.

```
SELECT customer_id,
       order_id,
       order_date,
       total_amount,
       ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY total_amount DESC
       ) AS rank_within_customer
FROM orders
ORDER BY customer_id, rank_within_customer;
```

Scroll through the results. Each customer's orders are numbered independently. Customer 1 has their own rank 1, 2, 3. Customer 2 has their own. This is PARTITION BY at work — it restarts the numbering for each group.

Now extend it: find only each customer's single highest-value order. Use a CTE to wrap this query, then filter for `rank_within_customer = 1`.

Query 7 — Overall revenue ranking with RANK

Rank all orders by total amount globally. Use both RANK and DENSE_RANK and compare the difference when there are ties.

```
SELECT order_id,
       customer_id,
       total_amount,
       RANK()       OVER (ORDER BY total_amount DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY total_amount DESC) AS dense_rank
FROM orders
ORDER BY total_amount DESC
LIMIT 15;
```

Look at any rows where two orders have the same total_amount. RANK will skip a number after a tie (1, 1, 3) while DENSE_RANK will not (1, 1, 2). In most business contexts DENSE_RANK is more useful — RANK creates confusing gaps that are hard to explain in a report.

Query 8 — Month-over-month revenue trend with LAG

Show monthly revenue and how it compares to the previous month — both as an absolute difference and as a percentage change.

```
WITH monthly_revenue AS (
    SELECT TO_CHAR(order_date, 'YYYY-MM')  AS month,
           SUM(total_amount)               AS revenue
    FROM orders
    WHERE status = 'delivered'
    GROUP BY TO_CHAR(order_date, 'YYYY-MM')
)
SELECT month,
       revenue,
       LAG(revenue) OVER (ORDER BY month)  AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY month)
                                            AS absolute_change,
       ROUND(
           100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
                 / NULLIF(LAG(revenue) OVER (ORDER BY month), 0),
           1
       )                                    AS pct_change
FROM monthly_revenue
ORDER BY month;
```

NULLIF(value, 0) prevents a division-by-zero error for the first month where LAG returns NULL. The first row will show NULLs for prev_month and the change columns — that is correct, there is no month before the first one. Look at the percentage changes — which month had the biggest jump? Which had a drop?

Query 9 — Customer value segmentation using CTEs

Classify every customer into a value tier based on their total spend, then summarise how many customers are in each tier and what percentage of total revenue each tier represents.

```
WITH customer_spend AS (
    SELECT c.customer_id,
           c.name,
           c.city,
           COALESCE(SUM(o.total_amount), 0) AS total_spent
    FROM customers c
    LEFT JOIN orders o
        ON c.customer_id = o.customer_id
        AND o.status = 'delivered'
    GROUP BY c.customer_id, c.name, c.city
),
customer_tiers AS (
    SELECT *,
           CASE
               WHEN total_spent > 30000 THEN 'VIP'
               WHEN total_spent > 10000 THEN 'High Value'
               WHEN total_spent > 0     THEN 'Active'
               ELSE 'Never Purchased'
           END AS tier
    FROM customer_spend
)
SELECT tier,
```

```
        COUNT(*)                                AS num_customers,
        ROUND(SUM(total_spent), 2)              AS tier_revenue,
        ROUND(
            100.0 * SUM(total_spent)
                  / NULLIF(SUM(SUM(total_spent)) OVER (), 0),
            1
        )                                       AS revenue_share_pct
FROM customer_tiers
GROUP BY tier
ORDER BY tier_revenue DESC;
```

Note the `SUM(SUM(total_spent)) OVER ()` in the last column — this is a window function inside an aggregate, which calculates the grand total across all groups so you can compute each tier's share. It looks unusual but it is a legitimate and useful pattern. Also note the LEFT JOIN with COALESCE — this ensures customers who have never placed an order still appear in the results with a spend of 0.

Query 10 — Session-to-purchase funnel analysis

For each customer who has both session data and order data, compare their browsing behaviour (from user_sessions) to their purchase behaviour. Find customers who browse a lot but buy rarely — a classic churn risk signal.

```
WITH session_summary AS (
    SELECT customer_id,
           COUNT(*)                            AS total_sessions,
           SUM(pages_viewed)                   AS total_pages,
           ROUND(AVG(duration_mins), 2)        AS avg_duration,
           SUM(CASE WHEN converted THEN 1 ELSE 0 END) AS converted_sessions
    FROM user_sessions
    GROUP BY customer_id
),
order_summary AS (
    SELECT customer_id,
           COUNT(*)            AS total_orders,
           SUM(total_amount)   AS total_spent
    FROM orders
    WHERE status = 'delivered'
    GROUP BY customer_id
),
combined AS (
    SELECT c.name,
           c.city,
           s.total_sessions,
           s.total_pages,
           s.avg_duration,
           s.converted_sessions,
           COALESCE(o.total_orders, 0) AS total_orders,
           COALESCE(o.total_spent, 0)  AS total_spent
    FROM session_summary s
    JOIN customers c ON s.customer_id = c.customer_id
    LEFT JOIN order_summary o ON s.customer_id = o.customer_id
)
```

```
SELECT name,
       city,
       total_sessions,
       total_pages,
       avg_duration,
       total_orders,
       total_spent,
       ROUND(
           CASE WHEN total_sessions > 0
               THEN 100.0 * total_orders / total_sessions
               ELSE 0
           END, 1
       ) AS orders_per_100_sessions
FROM combined
ORDER BY total_sessions DESC, orders_per_100_sessions ASC;
```

The orders_per_100_sessions column is a simple engagement-to-conversion ratio. A customer with many sessions but very few orders relative to their browsing is a candidate for a targeted discount or re-engagement campaign. This is the kind of feature you would compute in SQL before feeding it into a churn prediction model.

Save and Commit

```
cd ~/database-labs
git add lab3/queries.sql
git commit -m "Lab 3: Completed aggregations, window functions, and CTE
exercises"
git push origin main
```

Part 4 — Performance Awareness

Last week you ran EXPLAIN ANALYZE on a simple SELECT. This week, try it on a more complex query and pay attention to new things that appear in the output.

```
EXPLAIN ANALYZE
SELECT TO_CHAR(order_date, 'YYYY-MM') AS month,
       COUNT(*)                        AS num_orders,
       SUM(total_amount)               AS monthly_revenue
FROM orders
GROUP BY TO_CHAR(order_date, 'YYYY-MM')
ORDER BY month;
```

Look for these in the output:

HashAggregate. This is the operation PostgreSQL uses to perform GROUP BY. It builds a hash table in memory, groups rows into buckets, and computes aggregates for each. On large datasets with many groups, this can become a memory bottleneck.

Sort. ORDER BY triggers a sort operation. PostgreSQL shows you whether it sorted in memory or spilled to disk. Disk sorts are significantly slower.

Rows vs actual rows. PostgreSQL estimates how many rows each step will produce before running. Compare its estimate to the actual number. Large discrepancies are a sign the query planner may make poor decisions — this matters more as data grows.

For your deliverable, run EXPLAIN ANALYZE on Query 3 (monthly revenue) and Query 9 (customer segmentation CTE). Paste both outputs and write a short paragraph comparing them: which one has more operations? Which was slower? Why do you think the CTE query involves more steps?

Part 5 — AI Prompting for This Week's Concepts

Window functions and CTEs are the two concepts where students most often get stuck — and they are also the two where AI explanations are genuinely excellent if you ask the right questions.

Pattern 1 — Understanding PARTITION BY

The OVER clause with PARTITION BY is the part of window functions that confuses people most. Use this prompt to lock in the concept:

```
I am learning SQL window functions in PostgreSQL.

I understand that ROW_NUMBER() assigns a row number, but I am confused
about what PARTITION BY does. Here is a query I wrote:

SELECT customer_id,
       order_id,
       total_amount,
       ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY total_amount DESC
       ) AS rank_within_customer
FROM orders;

Please:
1. Explain PARTITION BY with a concrete analogy (not a technical definition)
2. Show me what the output would look like for 2 customers with 3 orders each
3. Show what changes if I remove PARTITION BY entirely
4. Explain the difference between ROW_NUMBER, RANK, and DENSE_RANK
   with an example where they give different results
```

Pattern 2 — Debugging a GROUP BY error

```
I am running this query in PostgreSQL and getting an error:

SELECT category, product_name, COUNT(*), AVG(price)
FROM products
GROUP BY category;

Error: ERROR: column "products.product_name" must appear in the
GROUP BY clause or be used in an aggregate function

My products table has: product_id, product_name, category, price, stock_qty

Please:
1. Explain exactly why this error occurs — what rule am I breaking?
2. Show two correct versions: one that includes product_name properly,
   and one that removes it
3. Explain the general rule for what can and cannot go in SELECT
   when you use GROUP BY
```

Pattern 3 — Building a CTE step by step

```
I need to write a complex SQL query and I want to use CTEs to keep
it readable. Here is my goal:

Find the top 3 customers by total spend, then for each of those
customers show their most recent order date and their average
order value.

My tables:
- customers (customer_id, name, city, signup_date)
- orders (order_id, customer_id, order_date, status, total_amount)

Please:
1. Break this into logical steps and explain each one before coding
2. Write the CTE version with comments explaining each CTE
3. Show me what each intermediate CTE produces (just describe the
   columns and a couple of example rows)
4. Then give me a challenge: a variation of this query I can try
   to write myself
```

Part 6 — Deliverables

Submit a single PDF named `Lab03_FirstNameLastName_RollNumber.pdf` to Google Classroom by the announced deadline. Late penalty: 25%.

Section 1 — 10 Queries (40 points)

For each query: formatted SQL code, screenshot of results from DBeaver or psql, and two to three sentences explaining what the output reveals about the data. This week the bar is higher than last week — do not just describe what the query does, describe what the result means in business terms. For example: "The results show that Electronics generates three times the revenue of Books despite having fewer products, suggesting it is the primary revenue driver."

Section 2 — Performance Report (20 points)

EXPLAIN ANALYZE output for Query 3 and Query 9. Written comparison: which query involved more operations? Which was slower? What new operation types appeared compared to last week's simple SELECT query?

Section 3 — AI Learning Log (30 points)

Minimum 3 entries using the same template from Lab 02. This week at least one entry must be about window functions or CTEs — the two new concepts. The entry should show that you experimented: you took AI's explanation, modified the query, predicted the result, ran it, and confirmed or corrected your prediction.

Section 4 — Reflection (10 points)

150 words minimum. What is the difference between WHERE and HAVING in your own words? When would you use a window function instead of GROUP BY? What part of this week took you the longest to understand and what finally made it click?

Quick Reference

```
-- ── Aggregate functions ──────────────────────────────────
COUNT(*)                    -- count all rows including NULLs
COUNT(column)               -- count non-NULL values only
SUM(column)                 -- total
AVG(column)                 -- mean
MIN(column)                 -- smallest value
MAX(column)                 -- largest value

-- ── GROUP BY skeleton ────────────────────────────────────
SELECT column, AGG_FUNCTION(other_column)
FROM table
WHERE row_level_filter          -- runs before grouping
GROUP BY column
HAVING group_level_filter       -- runs after grouping
ORDER BY ...;

-- ── Window function skeleton ─────────────────────────────
FUNCTION() OVER (
    PARTITION BY column     -- restart for each group
    ORDER BY column DESC    -- order within the group
)

-- ── Common window functions ──────────────────────────────
ROW_NUMBER()                -- unique sequential number, no ties
RANK()                      -- tied rows get same rank, gaps after
DENSE_RANK()                -- tied rows get same rank, no gaps
LAG(column, 1)              -- value from the previous row
LEAD(column, 1)             -- value from the next row
SUM(column) OVER (...)      -- running total within window

-- ── CTE skeleton ─────────────────────────────────────────
WITH
cte_one AS (
    SELECT ...
    FROM ...
    WHERE ...
),
cte_two AS (
    SELECT ...
    FROM cte_one        -- can reference previous CTEs
    WHERE ...
)
SELECT *
FROM cte_two
ORDER BY ...;

-- ── Useful functions this week ───────────────────────────
TO_CHAR(date_col, 'YYYY-MM')     -- format date as month string
EXTRACT(YEAR FROM date_col)       -- pull part of a date
NULLIF(value, 0)                  -- return NULL if value equals 0
                                  -- prevents division-by-zero
COALESCE(value, fallback)         -- return fallback if value is NULL
ROUND(value, decimal_places)      -- round a number
```